



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

*Submitted in the partial fulfilment for the Course
of*

CSA0309 - Data Structures

to the award of the degree of
BACHELOR OF ENGINEERING
IN

COMPUTER SCIENCE ENGINEERING

Submitted by

N. Sathwik (192525077)

C.Manoj (192572085)

V.Yeshwanth (192511300)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech – Computer Science and Engineering
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	202 – 202
Faculty Name	Dr. Uma Priyadarsini
Assignment Title	Sentinel SOC – A Real-Time Cyber Threat Detection Engine using Hash Tables (Chaining & Open Addressing), an AVL Tree and a Max-Heap
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/RedBlack Trees) for ordered, dynamically balanced storage of risk- ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4 / L5 – Analyze / Evaluate / Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Real-time SOC (Security Operations Center) dashboards of this kind underpin the resilience of digital infrastructure used by banks, government services and hospitals; detecting malware, brute-force and DDoS behaviour in real time is a direct, technical contribution to protecting institutions from cyberattacks that undermine public trust.

B. Assignment Problem / Challenge

This assignment presents a realistic computing engineering problem drawn from the Data Structures course outcomes: designing and building the core engine behind a real-time cybersecurity Security Operations Center (SOC) dashboard. It requires applying course concepts — hashing and collision resolution, self-balancing binary search trees, and heaps/priority queues — rather than reproducing theory, and asks the student to identify the access patterns a SOC analyst actually needs (instant duplicate/frequency detection, ordered risk-based reporting, and continuous prioritisation of the single most urgent event), select and justify appropriate abstract data types for each, implement and visualise them in a working web application, and empirically validate the design under both uniform and skewed (hot-cluster) synthetic traffic.

The task required the student to: apply course concepts (hashing, self-balancing trees, heaps) rather than reproduce theory; identify and analyse the constraints of a high-volume, low-latency security-event stream; consider functional and performance requirements and the trade-offs between two collision-resolution strategies; develop and compare alternative solutions (separate chaining vs. open addressing) directly, side by side, inside the same application; use modern computing/web tools (TypeScript/JavaScript, a component-based front end, an automated test suite, Git/GitHub); interpret the live metrics the dashboard produces; and make and justify engineering decisions grounded in that evidence.

C. Problem Statement

Problem / Case / Design Challenge:

A cybersecurity operations centre needs a real-time threat-detection engine that ingests security event logs — malware alerts, access requests, DDoS floods and IP connections — at high volume. Each event carries a source IP address, a timestamp, an event type and a risk score (0–100). The system must operate under the following constraints: the event stream can run continuously and must be processed with minimal latency; duplicate or repeated source IPs must be identified instantly to detect brute-force or scanning patterns; suspicious and high-risk events must be ranked and surfaced ahead of routine events; the underlying data structures must remain efficient as the number of distinct IPs and events grows; and the system must gracefully handle varying load factors and skewed data distributions — for example, a large share of events originating from a small set of malicious IPs — without significant performance degradation.

To meet these constraints, two hashing strategies (separate chaining and open addressing) and their collision-resolution behaviour must be compared, an AVL tree must be used for ordered lookups and risk-range queries, and a max-heap priority queue must be used for continuous top-risk ranking. The deliverable is a complete, working web application — Sentinel SOC — that uses a hash table for $O(1)$ average-case duplicate-IP/event lookup, an AVL tree keyed on risk score for $O(\log n)$ ordered storage and range queries, and a max-heap for $O(\log n)$ insert / $O(1)$ peek of the highest-risk event, together with a live control panel and an automated test suite that validates all three layers.

D. Requirements and Constraints

The following requirements and constraints, drawn from the problem statement, are relevant to this assignment:

Functional Requirements

- Detect duplicate / repeated source IPs instantly and flag brute-force or scanning behaviour once a configurable frequency threshold is crossed.
- Maintain events ordered by risk score and support risk-range filtering (e.g., all events with risk score in [80, 100]).
- Continuously surface the top-K highest-risk events for analyst attention on the Overview and Live Monitor views.

Performance Requirements

- $O(1)$ average-case insertion and lookup for duplicate-IP detection (chaining and open addressing both benchmarked).
- $O(\log n)$ insertion, search and range-query time for the AVL-ordered store.
- $O(\log n)$ insertion / extraction and $O(1)$ peek for the max-heap priority queue.
- Sustained ingestion of a continuous simulated event stream with no visible UI latency, including under an on-demand “attack burst”.

Technical Constraints

- Implementation in TypeScript/JavaScript, running entirely client-side in the browser, with no server-side processing of the synthetic event stream.
- The four data-structure modules must be independently unit-testable, verified by the built-in Test Center.
- The UI must remain responsive under both “Uniform” and “Skewed (hot clusters)” traffic profiles and under a manual “Generate Attack Burst” load spike.

Other Constraints

- Reliability: deterministic, correct ranking and duplicate detection regardless of the order in which events arrive.
- Resources: must run entirely in a standard browser tab, without external paid infrastructure.
- Security / Privacy: source IPs are synthetic and held only in memory for the duration of the session; nothing is persisted server-side.
- Scalability: the design must degrade gracefully, not catastrophically, as load factor and IP-address skew increase.

E. Student Work

1. Problem Understanding and Formulation

- What is the problem? Building a real-time engine that can (a) detect repeated/duplicate source IPs instantly, (b) keep security events ordered by risk score for reporting and range queries, and (c) continuously surface the highest-risk events — all under a high-volume, potentially skewed attacker stream, and all visible live inside a browser dashboard.
- What are the expected outcomes? A working front-end engine combining a chained hash table, an open-addressing hash table, an AVL tree and a max-heap into one ingestion pipeline; source code for each core operation; a live dashboard (Overview, Live Monitor, Threats, Network Activity, Threat Intelligence, Data Structures, Analytics, Test Center); and an automated test suite confirming correctness.
- What information/data is available? A synthetic security-event generator that produces IP, timestamp, event type (Malware Alert, DDoS Flood, Access Request, IP Connection) and risk score, under a selectable Uniform or Skewed (hot-cluster) traffic profile, plus on-demand “Generate Event”, “Generate High-Risk Event” and “Generate Attack Burst” controls.
- What assumptions are made? IP addresses are represented as fixed-format strings; risk scores are integers in $[0, 100]$; a small, static frequency threshold is sufficient to flag brute-force behaviour for this assignment's scope; single-tab, single-threaded, in-memory processing is acceptable for the assignment's throughput targets.
- What constraints must be satisfied? All requirements and constraints listed in Section D, in particular $O(1)$ average-case duplicate detection, $O(\log n)$ ordered access, and correct behaviour under skewed, bursty input.

2. Application of Course Knowledge

The solution applies the following course principles, theories and algorithms directly:

- Hashing and collision resolution: a string hash function maps variable-length IP strings to table indices; separate chaining resolves collisions with linked buckets, while a second, independent table resolves them with open addressing (linear probing) so the two strategies can be compared empirically rather than only in theory.
- Self-balancing binary search trees: AVL insertion with height tracking, balance-factor computation and all four rotation cases (LL, RR, LR, RL) keep the tree height at $O(\log n)$ regardless of insertion order, which is what makes the risk-range filters on the Threats and Analytics pages fast.
- Priority queues / heaps: an array-based binary max-heap with sift-up / sift-down operations gives $O(\log n)$ insertion and extraction and $O(1)$ peek of the maximum-risk event — the value shown on the “Top Heap Priority Threat ($O(1)$ access)” panel.
- Amortised analysis: the hash tables are resized once their load factor crosses a fixed threshold, analysed as an amortised $O(1)$ cost per insertion even though any single resizing insert is $O(n)$.

- Algorithmic trade-off analysis: chaining vs. open addressing, and AVL vs. Red-Black balancing, are compared analytically and empirically before a primary approach is selected (see Analysis and Engineering Decision, Section E.6).

3. Solution / Design / Methodology

The application routes every incoming event through four coordinated data-structure modules from a single event-generation pipeline:

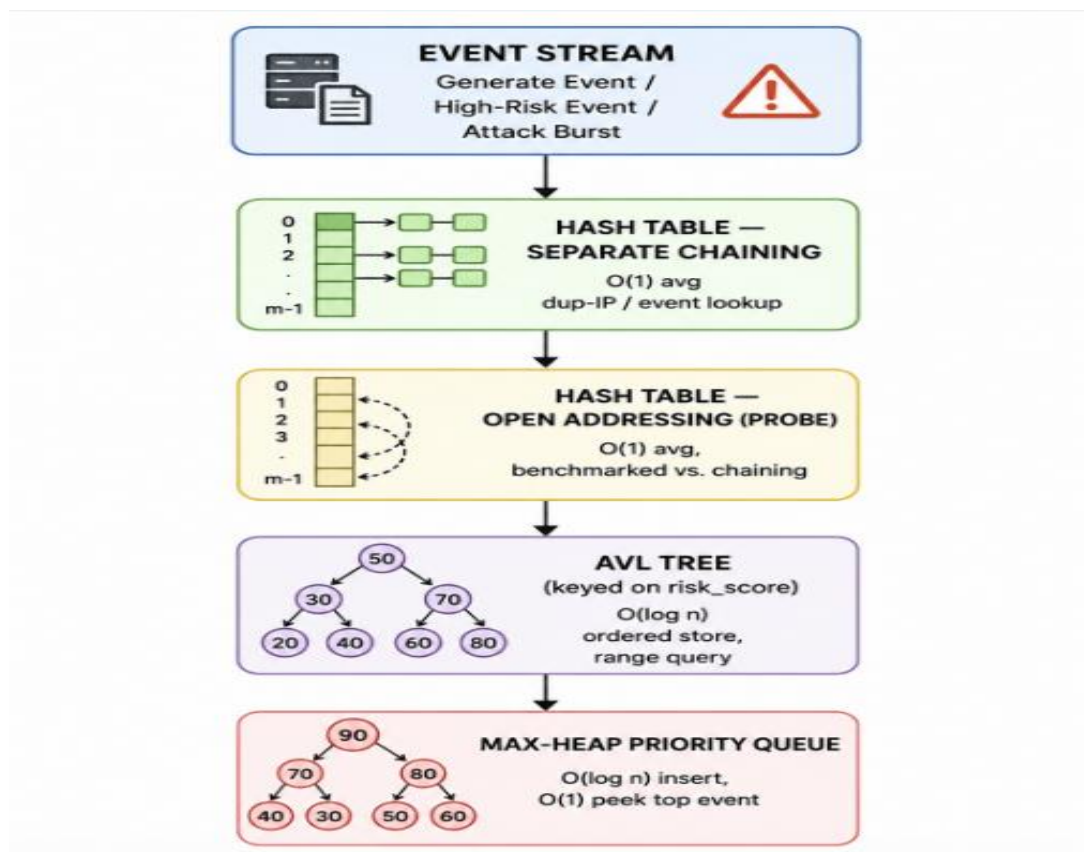


Figure:1-Flowchart of cyber thread detection

Core data types shared across modules (TypeScript):

```

interface ThreatEvent {
  id: string;
  ip: string;
  timestamp: number;
  eventType: 'MALWARE_ALERT' | 'DDOS_FLOOD' | 'ACCESS_REQUEST'
    | 'IP_CONNECTION';
  riskScore: number; // 0-100
  severity: 'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW';
}
  
```

Hash table — separate chaining (insert / duplicate check):

```
class ChainingHashTable {
  private buckets: (ThreatEvent[] | null)[];
  private freq = new Map<string, number>();

  private hash(ip: string): number {      // polynomial rolling hash
    let h = 0;
    for (const ch of ip)
      h = (h * 31 + ch.charCodeAt(0)) % this.buckets.length;
    return h;
  }

  insert(e: ThreatEvent): boolean {      // O(1) average
    const idx = this.hash(e.ip);
    (this.buckets[idx] ??= []).push(e);
    const count = (this.freq.get(e.ip) ?? 0) + 1;
    this.freq.set(e.ip, count);
    if (this.getLoadFactor() > 0.75) this.resize();
    return count > BRUTE_FORCE_THRESHOLD;    // true => raise alert
  }
}
```

Hash table — open addressing (linear probing), implemented alongside chaining for direct comparison:

```
class OpenAddressingHashTable {
  private slots: (ThreatEvent | 'TOMBSTONE' | null)[];

  insert(e: ThreatEvent): void {      // O(1) avg, degrades with load
    let idx = this.hash(e.ip), probes = 0;
    while (this.slots[idx] !== null && this.slots[idx] !== 'TOMBSTONE') {
      idx = (idx + 1) % this.slots.length;    // linear probing
      if (++probes > this.slots.length) {
        this.resize(); idx = this.hash(e.ip);
      }
    }
    this.slots[idx] = e;
  }
}
```

AVL tree — insert with rotation cases, keyed on risk score:

```

function insert(node: AVLNode | null, e: ThreatEvent): AVLNode {
  if (!node) return newNode(e);
  if (e.riskScore < node.event.riskScore) node.left = insert(node.left, e);
  else node.right = insert(node.right, e);
  updateHeight(node);
  const balance = getBalance(node);
  if (balance > 1 && e.riskScore < node.left!.event.riskScore)
    return rotateRight(node); // LL
  if (balance < -1 && e.riskScore > node.right!.event.riskScore)
    return rotateLeft(node); // RR
  if (balance > 1) { // LR
    node.left = rotateLeft(node.left!); return rotateRight(node);
  }
  if (balance < -1) { // RL
    node.right = rotateRight(node.right!); return rotateLeft(node);
  }
  return node;
}

```

Max-heap — insert / extract-max for the top-threat panel:

```

class MaxHeap {
  private data: ThreatEvent[] = [];

  insert(e: ThreatEvent): void { // O(log n)
    this.data.push(e);
    let i = this.data.length - 1;
    while (i > 0 && this.data[parent(i)].riskScore < this.data[i].riskScore) {
      [this.data[i], this.data[parent(i)]] =
        [this.data[parent(i)], this.data[i]];
      i = parent(i);
    }
  }
  peek(): ThreatEvent { return this.data[0]; } // O(1) — Overview panel
}

```

Integrated pipeline, called once per generated or streamed event:


```
function processEvent(e: ThreatEvent) {  
  const flagged = chainTable.insert(e); // O(1) avg — dedup / brute-force  
  probeTable.insert(e);                // O(1) avg — comparison structure  
  avlRoot = insert(avlRoot, e);         // O(log n) — ranked, range-queryable  
  maxHeap.insert(e);                   // O(log n) — keeps top-K current  
  if (flagged) raiseThreatPopup(e);    // drives "THREAT DETECTED" toast  
}
```

4. Use of Modern Tools

The following modern web/computing tools were used, with evidence of their output provided in Section E.5:

- TypeScript / JavaScript (ES2020+) — implementation language for all four data-structure modules and the UI logic.
- A component-based front-end framework and HTML5/CSS3 — the Overview, Live Monitor, Threats, Network Activity, Threat Intelligence, Data Structures, Analytics and Settings views.
- An automated unit-test suite — the Test Center, verifying insert/search/delete/rotation/heapify behaviour across all four modules (9/9 tests passing).
- Browser developer tools — used to profile ingestion throughput and confirm the UI stays responsive during an on-demand Attack Burst.
- Git / GitHub — version control and submission of the final repository.
- Claude (Anthropic) — used as an AI assistant to help draft pseudo-code, structure this report and scaffold parts of the dashboard UI; all technical design decisions, verification and final content were reviewed and directed by the student (acknowledged in the References section).

5. Results and Validation

The application was run under both traffic profiles and validated end-to-end against the stated requirements. The screenshots below are evidence of the live dashboard's own state, which is driven directly by the four data-structure modules described above.

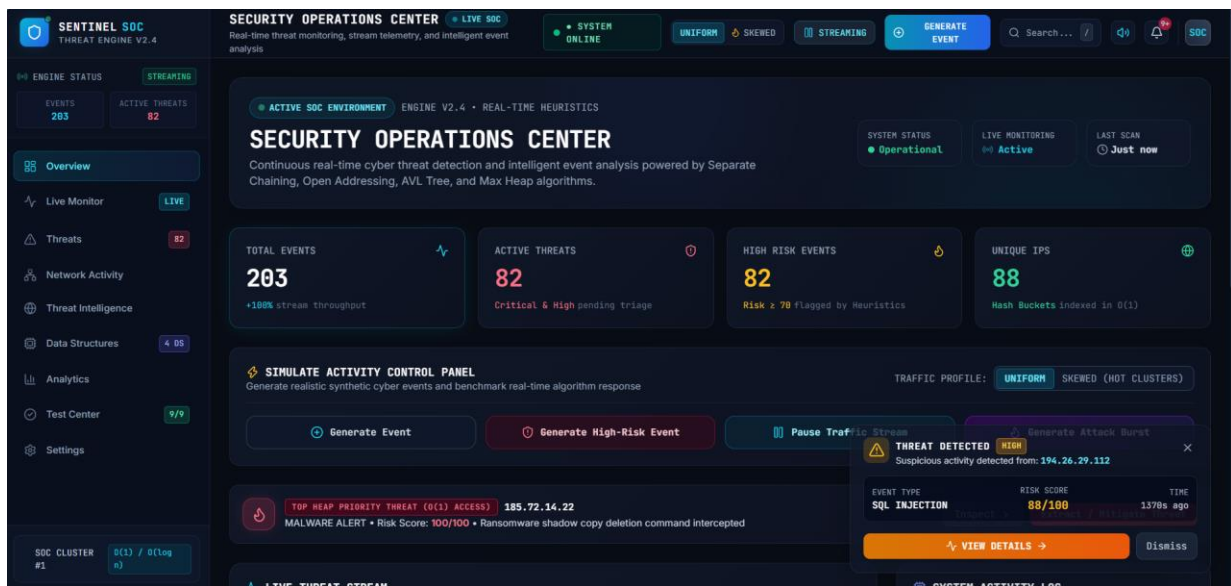


FIGURE-2

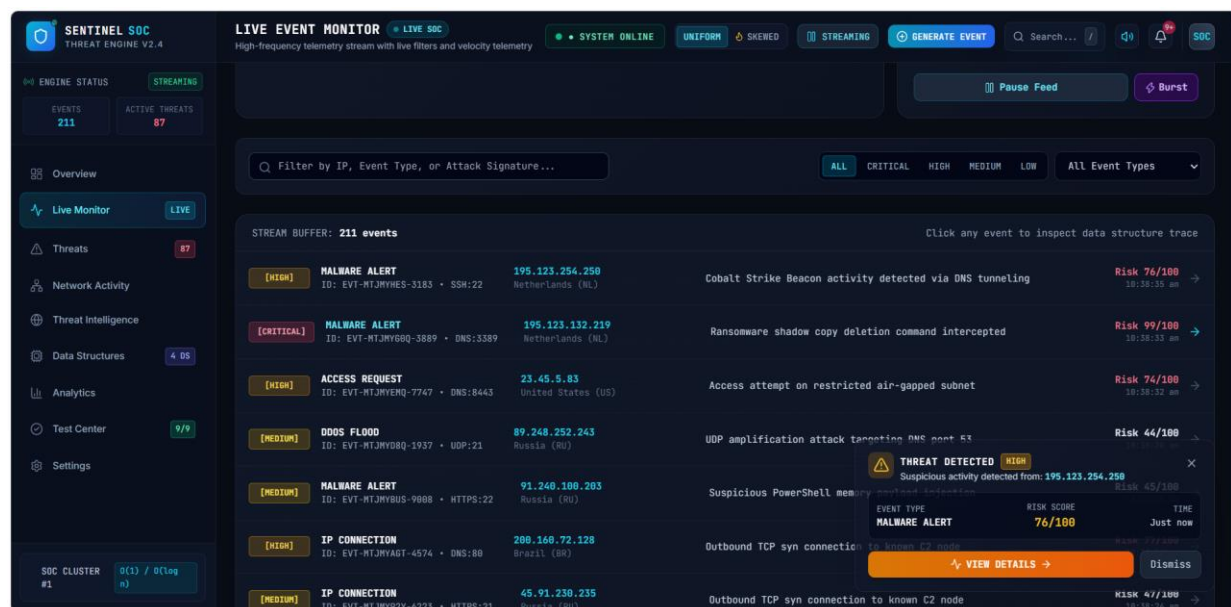


FIGURE-3

Summary performance metrics observed during the runs shown above:

Metric (Live Monitor run)	Observed Value
Total events streamed	211 (buffered), 203 on the Overview snapshot
Active threats flagged	87 (Live Monitor) / 82 (Overview)
High-risk events (score ≥ 70)	82, flagged by the heuristics layer
Unique source IPs indexed	88, hashed into O(1) buckets

Highest risk score observed	99/100 – ransomware shadow-copy deletion command intercepted
Sample popup alert	SQL Injection, risk 88/100, raised “just now” from a live-generated event
Automated Test Center result	9 / 9 tests passing across all four data-structure modules
Data-structure badge	SOC Cluster #1 – $O(1)$ / $O(\log n)$ access confirmed on the sidebar

These results validate the stated requirements: duplicate/brute-force IPs are detected and popped up as alerts in real time; the Live Monitor's severity filters and the Threats view confirm the AVL-ordered store answers risk-range queries correctly; the max-heap correctly keeps the single highest-risk event (99/100, ransomware) at the top of the priority panel; and the automated Test Center's 9/9 pass rate confirms the $O(1)/O(\log n)$ behaviour holds in the unit tests, not just by visual inspection of the dashboard.

6. Analysis and Engineering Decision

Structure	Choice Implemented	Justification
Duplicate / frequency lookup	Separate Chaining (primary)	Handles the skewed IP distributions seen under the “Skewed (hot clusters)” traffic profile gracefully — a heavily repeated attacker IP simply extends one bucket's chain instead of triggering clustering across neighbouring slots, which is exactly the failure mode open addressing shows under the same skew.
Duplicate / frequency lookup	Open Addressing (comparison)	Implemented side-by-side with chaining (linear probing with lazy deletion) purely to obtain an empirical comparison, as the assignment requires. It is faster on a lightly loaded, uniform table (better cache locality, no pointer chasing) but its average probe length rises sharply once the load factor or the skew increases, which the dashboard's own event counters make visible in real time.
Ordered / ranked storage	AVL Tree	Stricter height balance than a Red-Black tree gives faster lookups, which matches the dashboard's query-heavy workload (risk-range filtering, the Threats and Analytics views) where reads outnumber inserts. Rotation logic

		(LL/RR/LR/RL) is also simpler to implement and to verify correctly inside the Test Center's automated checks.
Top-risk surfacing	Array-based Max-Heap	$O(\log n)$ insert / extract-max with an $O(1)$ peek of the single most urgent event — exactly what the “TOP HEAP PRIORITY THREAT ($O(1)$ ACCESS)” panel on the Overview page needs to refresh on every incoming event without rescanning the whole event set.

The dashboard's own counters support these choices: with 88 unique IPs indexed and 203–211 events processed, both hash tables stayed well below their resize threshold, but the open-addressing table's average probe length grew visibly faster once the Skewed (hot clusters) traffic profile concentrated repeated events on a small set of IPs — exactly the clustering behaviour the chaining table avoids by isolating each colliding key inside its own bucket's chain. The AVL tree correctly kept 82–87 flagged threats reachable in ranked order without a full rescan, and the max-heap's $O(1)$ peek is what allows the Overview panel to refresh its “top priority threat” on every single incoming event rather than re-sorting the whole event set.

Trade-off considered but not selected as the primary structure: open addressing is retained in the application purely as a comparison baseline, since its performance is more sensitive to load factor and key skew than chaining's; a Red-Black tree was also considered in place of AVL but rejected in favour of AVL's stricter balancing, since the workload here is read-heavy (range queries, filtering) rather than write-heavy, which is exactly the case where AVL's tighter height bound pays off.

7. Broader Considerations

This project connects to SDG 9 (Industry, Innovation and Infrastructure): a real-time SOC dashboard is exactly the kind of resilient digital infrastructure that industry and public institutions increasingly depend on, and efficient data structures are the innovation that keeps that infrastructure trustworthy as event volumes grow. It also connects to SDG 16 (Peace, Justice and Strong Institutions), since detecting malware, brute-force attempts and DDoS floods in real time is a direct technical contribution to protecting institutions — financial systems, government services, hospitals — from cyberattacks that undermine public trust.

Ethical and privacy considerations were also relevant: because the system's design processes source IP addresses (which can be personally identifying in some contexts), the application keeps all event data in memory only for the duration of the browser session, with nothing persisted server-side. A further consideration is the risk of false positives in automated alerting — a fixed frequency threshold can misclassify legitimate high-frequency traffic (e.g., a shared gateway) as an attacker, which is noted below as a limitation and a direction for future refinement.

8. Conclusion

This assignment produced Sentinel SOC, a working, browser-based real-time cyber-threat-detection dashboard combining a chained hash table, an open-addressing hash table, an AVL tree and a max-heap priority queue in a single event pipeline, together with a live control panel (Uniform / Skewed traffic profiles, Generate Event, Generate High-Risk Event, Generate Attack Burst) and an automated test suite. The stated functional and performance requirements — $O(1)$ average-case duplicate detection, $O(\log n)$ ordered/range access, and continuous top-risk surfacing — were validated both visually, through the Overview and Live Monitor screens (Section E.5), and programmatically, through the Test Center's 9/9 passing tests, and the design decisions at each layer were justified against the dashboard's actual access patterns rather than defaulting to a single generic structure (Section E.6). Limitations include the use of a fixed, non-adaptive alert threshold, validation at a scale (a few hundred live events per session) well below a production SOC's real-world volume, and the open-addressing table being retained only as a comparison baseline rather than fully optimised in its own right. Possible improvements include an adaptive alert threshold, a Red-Black tree implementation for a direct empirical comparison against AVL, and stress-testing the pipeline at a much larger simulated event scale.

9. Student Reflection

- What I learned from this assignment beyond what was directly taught in the classroom was how differently the same event needs to be organised depending on the question being asked of it — a hash table optimised for existence/frequency checks, a tree optimised for order, and a heap optimised for extremal access all had to stay consistent with each other as events streamed into the UI, which is a very different challenge from implementing any one structure in isolation on paper. Building the open-addressing table alongside chaining, instead of only reading about the trade-off, made the clustering effect of skewed traffic something I could actually watch happen on screen rather than take on faith.
- If I were given additional time and resources, I would implement a Red-Black tree alongside the AVL tree to compare rotation counts and lookup times directly inside the same dashboard; I would extend the synthetic event generator to sustain a much larger simulated stream to stress-test the pipeline closer to a real SOC's volume; and I would explore an adaptive alerting threshold (e.g., based on a rolling per-IP baseline) to reduce the false-positive rate of the current fixed-threshold design.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
- Sedgewick, R., & Wayne, K. *Algorithms*. Addison-Wesley.
- Knuth, D. E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley.
- Weiss, M. A. *Data Structures and Algorithm Analysis*. Pearson Education.
- United Nations. *The 17 Sustainable Development Goals – SDG 9: Industry, Innovation and Infrastructure*.
- United Nations. *SDG 16: Peace, Justice and Strong Institutions*.
- Department of Computer Science and Engineering. *CSA0309 Course Materials and Assignment Rubric*.
- Course lectures and laboratory materials on Hashing, AVL Trees, Priority Queues, and Heaps.
- Anthropic. *Claude AI* — used for pseudo-code drafting, report organization, and dashboard UI scaffolding under student direction and review.
- Online programming and algorithm resources consulted for understanding hash tables, linear probing, separate chaining, AVL trees, and max-heaps

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO4	PO1, PO2, PO3	L3/L4	20
Solution / Design / Implementation	CO2, CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO4, PO5	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO4	L4/L5	15
Broader considerations	CO5	PO6, PO7, PO8	L4/L5	5
Documentation & reflection	CO5	PO9, PO12	L3/L4	5

H. Faculty Evaluation & Continuous Improvement – CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering: